Time-Window Pricing and Schedule Rigidity

Macharia Barii

November 3, 2026

Abstract

Without time windows, route pricing is an allocation problem: divide shared cost among drops by service time and distance. With time windows, it becomes a feasibility problem first and an allocation problem second. A route with conflicting windows is not expensive—it is impossible, and no price can make it work. And when it is feasible, the allocation rule changes, because a narrow window consumes a scarce resource that a wide window does not: schedule flexibility. This paper derives the feasibility gate, the schedule window (the range of arrival times that keeps the route feasible), the three-component allocation rule with a rigidity term, the worked CBD → Thika example with three windows, the window menu, the four responses to infeasibility, and the connection to the bucket system and the flywheel.

1 The feasibility gate (before any pricing)

A route with drops $1, \ldots, n$ in some sequence is feasible if there exist arrival times $t_1, \ldots, t_n$ satisfying:

$$e_j \le t_j \le \ell_j \quad \text{for all } j \tag{1}$$

$$t_{j+1} \ge t_j + \mu_j + \tau_{j,j+1} \quad \text{for all consecutive } j \tag{2}$$

where $[e_j, \ell_j]$ is the drop's window, μ_j its service time, and $\tau_{j,j+1}$ the travel time between stops.

This is a system of difference constraints, solvable in $O(n^2)$ by forward/backward pass. If no schedule exists, the route is infeasible and no price makes it feasible. The correct response is structural: reorder the drops, split the route, widen a window, or reject a drop.

The rule: never price an infeasible route. Check feasibility first. This is Gate 5 (schedulability) of the drop proof rule, elevated to the route level.

2 The schedule window and the forward/backward pass

For a feasible route, each drop has not just its stated window but a *schedule window*: the range of arrival times at that drop that keeps the entire route feasible.

Forward pass (earliest arrival):

$$E_j = \max\left(e_j,\ E_{j-1} + \mu_{j-1} + \tau_{j-1,j}\right) \tag{3}$$

Backward pass (latest arrival):

$$L_j = \min\left(\ell_j,\ L_{j+1} - \mu_j - \tau_{j,j+1}\right) \tag{4}$$

The schedule window of drop j is $[E_j, L_j]$, with width $S_j = L_j - E_j$.

This is the quantity that drives the allocation. A drop with a narrow schedule window consumes the route's flexibility. A drop with a wide schedule window provides flexibility—it can absorb delays without breaking the route. The customer who gives you a wide window is doing you a favour, and should pay less.

Drop type	S_j	What it means
Flexible	Large	Can be served any time in a wide range without breaking the route
Critical	Small	Must be served at a specific time; constrains the whole route
Binding	Zero	Exactly one feasible arrival time; the route is fragile at this drop

3 The three-component allocation rule

The route's shared cost is allocated across three components, not two:

$$s_j = \alpha \cdot \underbrace{\frac{\mu_j}{\sum_k \mu_k}}_{\text{service share}} + \beta \cdot \underbrace{\frac{\Delta d_j}{D_R}}_{\text{distance share}} + \gamma \cdot \underbrace{\frac{1/S_j}{\sum_k 1/S_k}}_{\text{rigidity share}} \tag{5}$$

with $\alpha + \beta + \gamma = 1$. Default: $\alpha = 0.35$, $\beta = 0.25$, $\gamma = 0.40$.

Why the rigidity term is inverse schedule window: $1/S_j$ is large when S_j is small. A drop with a narrow schedule window (critical, rigid) gets a large share. A drop with a wide schedule window (flexible) gets a small share.

The interpretation: a tight window is like a bad address—it consumes a scarce resource (schedule flexibility) and should pay for it. A wide window is like a well-verified address—it makes the route easier to schedule and should be rewarded.

When there are no time windows: $S_j \to \infty$ for all drops, the rigidity shares become equal, and γ contributes a uniform term that can be absorbed. The formula reduces to the two-component rule from the previous answer.

4 Worked example: CBD → Thika with three windows

Route: Depot (CBD, 7:30 departure) → A (Kasarani, 12 km) → B (Ruiru, 10 km further) → C (Thika, 14 km further). Total: 36 km.

Inputs.

Drop	Window	μ_j	a	Δd_j
A	8:00–10:00 (120 min)	5 min	0.85	12 km
B	8:30–9:15 (45 min — tight)	20 min	0.50	10 km
C	9:30–12:00 (150 min — wide)	8 min	0.75	14 km

Travel: Depot→A = 20 min, A→B = 15 min, B→C = 25 min.

Step 1 — Feasibility check. Schedule: leave 7:30, arrive A 7:50 (wait 10 min), serve 8:00–8:05, arrive B 8:20 (wait 10 min), serve 8:30–8:50, arrive C 9:15 (wait 15 min), serve 9:30–9:38. All windows satisfied. Route is feasible. Total time: 7:30–9:38 = 128 min.

Step 2 — Schedule windows.

Drop	Earliest E_j	Latest L_j	S_j
A	8:10	8:55	45 min
B	8:45	9:15	30 min
C	9:30	12:00	150 min

B has the narrowest schedule window (30 min)—it is the most critical drop. C has the widest (150 min)—it is the most flexible.

Step 3 — Rigidity shares.

Drop	$1/S_j$	Rigidity share
A	0.0222	0.357
B	0.0333	0.535
C	0.0067	0.108

B carries 53.5% of the rigidity burden despite being only 1/3 of the route. C carries 10.8% because its window gives the route flexibility.

Step 4 — Combined shares. Using $\alpha = 0.35$, $\beta = 0.25$, $\gamma = 0.40$:

Drop	Service share	Distance share	Rigidity share	s_j
A	0.152	0.333	0.357	0.279
B	0.606	0.278	0.535	0.496
C	0.242	0.389	0.108	0.225

Step 5 — Prices. Shared cost $= c_0 + \Phi C_p + c_{\text{km}} D_R = 1{,}300 + 200 + 900 = 2{,}400$. Standalone: A = 200, B = 350, C = 250. All $\hat{p} \approx 0.9$, $A(a) \approx 1.1$.

Drop	Standalone	Allocated shared	Net n_j	Client price
A	200	$0.279 \times 2{,}400 = 670$	879	1,020
B	350	$0.496 \times 2{,}400 = 1{,}190$	1,556	1,805
C	250	$0.225 \times 2{,}400 = 540$	799	927

Step 6 — Compare to the no-window case.

The wide-window customer gets a 15% discount. The tight-window customer pays 25% more. This is correct—the tight window constrains the entire route, and the wide window gives it flexibility.

5 The window menu: pricing as a function of flexibility

The most important operational consequence: time windows are a pricing lever, not just a constraint. Offer the customer a menu, and let them choose how much flexibility to give you.

This is the airline model applied to last-mile. Advance purchase gets a discount because it gives the airline scheduling flexibility. Last-minute booking pays a premium because it consumes capacity that could have been sold. The same economics apply here: the customer who accepts a wide window is participating in temporal consolidation, and the discount is the fair sharing of the gains.

The flywheel effect: customers who accept wider windows raise the effective drop rate (less waiting, less rushing), which raises contribution, which feeds the loop. The window menu is not just a pricing tactic—it is a mechanism for shaping demand to fit the route structure.

6 When the route is infeasible: the four responses

If the feasibility check fails, pricing is irrelevant. The correct responses are structural:

The most common response is widen. A customer with a 45-minute window who is told "your window makes the route infeasible; a 2-hour window would make it work and save you 20%" will usually accept. The discount is cheaper than the alternative—losing the delivery entirely.

Drop	Price without windows	Price with windows	Change
A	813	1,020	+25% (moderately tight window)
B	1,835	1,805	−2% (already the largest payer)
C	1,096	927	−15% (wide window earns a discount)

Window width	Price multiplier	What it buys you
±4 hours (anytime)	0.80×	Maximum flexibility—the drop can fill any bucket
±2 hours	1.00×	Standard
±1 hour	1.20×	Constrained—must go on a specific bucket
±30 min	1.45×	Very tight—constrains the entire route
Exact time	1.75×	Dedicated—essentially a dedicated trip

7 Connection to the bucket system and the flywheel

In the bucket system: each bucket has a departure time. A drop with a wide window can go to any bucket in its corridor; a drop with a tight window must go to a specific bucket. The bucket assignment is itself a consolidation decision—assigning a tight-window drop to a bucket constrains the bucket's flexibility, which raises the cost of every other drop in that bucket. This is why the bucket system needs the schedule-window computation: it tells the dispatcher which drops are flexible and which are rigid before they are assigned.

In the flywheel: time windows affect the effective drop rate. A route with tight windows has more waiting time and less effective capacity, so R_{obs} falls relative to $R(a)$. This means the utilisation diagnostic must distinguish three cases:

The waiting time is the diagnostic that separates window-constraint from consolidation-constraint. A corridor with low utilisation and high waiting time is not demand-starved—it is schedule-constrained. The lever is the window menu, not client acquisition.

8 The extended pricing rule in one line

Check feasibility first—if the route cannot be scheduled, no price works. If it can, allocate shared cost across three components: service time, distance, and schedule rigidity (the inverse of the schedule window). The drop with the tightest window pays the most because it constrains the whole route; the drop with the widest window pays the least because it gives the route flexibility. And offer the customer a menu—a wider window is a discount, because it is temporal consolidation, and the discount is the fair sharing of the gains.

Response	When	What it means
Reorder	A different sequence makes the route feasible	Try TSPTW; often a small reordering fi
Split	Two routes can each be feasible	Route A serves drops 1–2; route B serve
Widen	The customer will accept a wider window for a discount	Offer the window menu—the discount is
Reject	No reordering, splitting, or widening works	The drop cannot be served on this route

$u = R_{\text{obs}}/R(a)$	Diagnosis	Lever
$u < 0.7$, high wait time	Window-constrained	Widen windows; offer temporal discounts
$u < 0.7$, low wait time	Consolidation-constrained	Acquire demand; extend batching window
$u > 0.9$	Addressability-constrained	Verify addresses; raise drop value